

Plano Consolidado de Implementacao

Auditoria completa do codebase, tres decisoes estrategicas e estrategia unificada em 3 blocos de execucao atomica.

SMARTHOTEL SaaS • 2026 • v2.0

Auditoria de 6 areas | 3 decisoes fundamentadas | 3 blocos de execucao | 23 arquivos mapeados | 12 testes de verificacao

ESTRATEGIA UNIFICADA

Sumario

1. Auditoria Completa do Codebase	2
1.1. Prisma Schema — Estado Atual	2
1.2. Infraestrutura BullMQ + Redis	2
1.3. API Routes — Estado Atual	3
1.4. Pipeline de Eventos — Arquitetura Atual	3
1.5. Ambiente e Dependencias	4
2. Tres Decisoies Estrategicas	5
2.1. Decisao 1: Escopo da Implementacao	5
2.2. Decisao 2: Coexistencia vs Migracao	5
2.3. Decisao 3: ActionLog vs Logs Existentes	6
3. Estrategia Consolidada de Implementacao	6
4. Mapa de Arquivos — Antes e Depois	8
5. Plano de Verificacao	9
5.1. Verificacao do Bloco A	9
5.2. Verificacao do Bloco B	9
5.3. Verificacao do Bloco C	9
6. Matriz de Riscos	10
7. Timeline Estimada	10

1. Auditoria Completa do Codebase

Esta auditoria foi conduzida analisando diretamente o código-fonte do repositório zehla-backend, confrontando cada componente descrito no documento ZEHla_Brain_Fase4_Implementacao_Codigo.pdf (38 páginas) com o estado real de implementação. O objetivo é mapear com precisão os gaps críticos e fornecer uma base sólida para a estratégia de implementação consolidada.

A auditoria revelou que o projeto possui uma fundação sólida em termos de modelos de dados básicos (Lead e Event) e uma pipeline de eventos síncrona funcional com 5 estágios. Porém, a infraestrutura de processamento assíncrono (BullMQ, Redis, Workers) está completamente ausente, e todos os 7 dispatchers de ação são placeholders que apenas registram logs no console. A base de dados é SQLite local, sem suporte a concorrência para escritas simultâneas de webhooks.

1.1. Prisma Schema — Estado Atual

O schema atual utiliza SQLite como banco de dados e possui 4 modelos: User, Post (boilerplate), Lead e Event. O modelo Lead possui 22 campos incluindo scoreQual, scoreValid, scoreTotal, funilEstagio e cluster. O modelo Event possui 17 campos incluindo tipo, fonte, dados (JSON), pontosGerados, scoreAntes e scoreDepois. Não existem modelos ActionLog, WebhookLog, AgentLog ou SecurityAlert. Os enums são definidos como constantes TypeScript em src/lib/events/types.ts, não como enums Prisma nativos.

Componente	Status	Detalhes
Lead Model	Parcial	22 campos, faltam: previousCluster, lastActivityAt, eventSource
Event Model	Parcial	17 campos, similar ao LeadEvent do PDF mas sem dedupHash e status
ActionLog Model	Ausente	Não existe. Gap crítico de governança.
WebhookLog Model	Ausente	Não existe. Sem rastreabilidade de integrações externas.
Prisma Enums	Ausente	Enums são constantes TS em types.ts, não enums Prisma.
Database	SQLite	Sem suporte a concorrência. Ideal para dev, problemático em prod.

Tabela 1: Status dos modelos Prisma

1.2. Infraestrutura BullMQ + Redis

Nenhum componente de processamento assíncrono existe no projeto. Não há conexão Redis, nenhum módulo de filas BullMQ, e nenhum worker background. Toda a lógica de processamento de eventos roda de forma síncrona dentro do handler POST /api/events/route.ts. Isso significa que cada requisição de webhook bloqueia a thread do servidor até que todos os 5 estágios da pipeline sejam executados sequencialmente. Em produção com alto volume, isso causaria timeouts, perda de eventos e degradação de performance generalizada.

Componente	Status	Impacto
lib/redis.ts	Ausente	Sem message broker para processamento assincrono
lib/queues.ts	Ausente	Filas BullMQ nao definidas (0 de 5)
BullMQ Workers	Ausentes	0 workers implementados
Worker Orchestrator	Ausente	Sem graceful shutdown ou gerenciamento centralizado
ioredis / bullmq	Ausente	Dependencias nao instaladas
Rate Limiting	Ausente	Sem protecao contra abuso de API
Deduplicacao	In-memory	Cache perdido em restart, inutil com multi-instancia

Tabela 2: Status da infraestrutura de filas

1.3. API Routes — Estado Atual

O projeto possui 6 endpoints funcionais. O endpoint principal e POST /api/events que implementa a pipeline de 5 estagios de forma sincrona. Existe tambem GET /api/events/stats com metricas reais calculadas do banco, GET /api/leads com CRUD completo e filtros, GET /api/leads/analytics com analises de distribuicao, e POST /api/leads/seed para carga de dados. Porem, faltam endpoints dedicados para tracking passivo (pixel), webhook generico externo e metricas do ZEHLA Brain especificas.

Endpoint	Metodo	Status	Gap
/api/events	POST+GET	Funcional	Pipeline sincrona, nao assincrona via fila
/api/events/stats	GET	Funcional	Metricas reais existem, bom ponto de partida
/api/leads	CRUD	Funcional	Completo com filtros e agregacoes
/api/leads/analytics	GET	Funcional	Distribuicao UF/regiao/score
/api/events/track	POST	Ausente	Endpoint generico de captura de eventos
/api/events/webhook	POST	Ausente	Webhook receiver com HMAC + mapeamento
/api/brain/metrics	GET	Ausente	Metricas dedicadas do ZEHLA Brain
/api/track/click/*	GET	Ausente	Tracking pixel de email e redirect de clique

Tabela 3: Status dos endpoints API

1.4. Pipeline de Eventos — Arquitetura Atual

A pipeline de eventos existe em src/lib/events/ como um conjunto de 7 modulos TypeScript que executam de forma sincrona dentro do handler POST /api/events. O fluxo e: o controller recebe o payload, chama pipeline.ts que orquestra os 5 estagios sequencialmente (captura, validacao, enriquecimento,

classificacao, acao), persiste o Event no Prisma e atualiza o Lead. O modulo actions.ts contem 7 dispatchers de acao, mas todos sao placeholders que retornam status "planejada" e apenas fazem console.log. O modulo scoring.ts contem logica funcional de calculo de score, cluster e inferencia de dor, que sera reaproveitada nos workers.

Modulo	Arquivo	Funcao	Status
Tipos	types.ts	8 tipos de evento, scores, clusters, cores	Funcional
Validador	validator.ts	Validacao de payload + fingerprint dedup	Funcional
Enriquecedor	enricher.ts	Lookup/criacao de lead + historico	Funcional
Scorer	scoring.ts	Score, cluster, funil, dor, acao recomendada	Funcional
Acoes	actions.ts	7 dispatchers (email, WhatsApp, reuniao)	Placeholders
Orquestrador	pipeline.ts	Coordena estagios 1-5 + persistencia	Funcional (sync)
Index	index.ts	Barrel re-export	Funcional

Tabela 4: Modulos da pipeline de eventos atual

Gap Critico: Os 7 action dispatchers em actions.ts sao todos placeholders. Eles retornam status "planejada" e fazem apenas console.log. Nenhuma integracao real com email (Resend), WhatsApp (Z-API/Twilio) ou CRM esta implementada.

1.5. Ambiente e Dependencias

O ambiente atual e extremamente minimalista. A unica variavel de ambiente configurada e DATABASE_URL apontando para SQLite local. Nao existem chaves de API para nenhum servico externo. As unicas dependencias relevantes instaladas sao Prisma (v6.11.1), Next.js (v16.1.1), Zod (v4.0.2), Zustand (v5.0.6), Recharts (v2.15.4) e Leaflet (v1.9.4). Nao ha BullMQ, ioredis, Resend, Twilio, n8n SDK ou qualquer biblioteca de processamento de filas, email ou mensageria. O package.json nao contem scripts de producao ou configuracao de PM2.

Item	Status
DATABASE_URL (SQLite)	Configurado
REDIS_URL	Ausente
RESEND_API_KEY	Ausente
ZAPI_URL / ZAPI_TOKEN	Ausente
WEBHOOK_SECRET	Ausente
NEXTAUTH_SECRET	Ausente

bullmq package	Nao instalado
ioredis package	Nao instalado
resend package	Nao instalado
ecosystem.config.js	Ausente
docker-compose.yml	Ausente

Tabela 5: Status do ambiente e dependencias

2. Tres Deciso es Estrategicas

Baseado na auditoria completa do codebase, as seguintes decisoes foram tomadas. Cada decisao e fundamentada no estado real do projeto e nas restricoes tecnicas identificadas, garantindo que a estrategia de implementacao seja pragmatica, de baixo risco e maximamente reaproveitavel.

2.1. Decisao 1: Escopo da Implementacao

DECISAO: Execucao Consolidada (Fase Unica) ao inves de 5 Fases Sequenciais

Racional: A auditoria revelou que nao existe nenhum worker, nenhum BullMQ, nenhum Redis no codebase. Nao ha workers para migrar nem conflitos de arquitetura. A pipeline de 5 estagios existe como codigo sincrono em `src/lib/events/` e pode ser "ejetada" para workers BullMQ com modificacoes minimas. Separar em 5 fases sequenciais criaria dependencias artificiais (ex: criar as filas na Fase 1 mas so usar na Fase 2, deixando codigo morto por dias).

Estrategia: Executar tudo em uma unica onda de implementacao organizada em 3 blocos de trabalho sequenciais internamente mas entregues como resultado atomico:

- **Bloco A — Fundacao (2-3h):** Instalar dependencias (bullmq, ioredis), criar `lib/redis.ts` e `lib/queues.ts`, atualizar Prisma schema (ActionLog, WebhookLog, campos novos no Lead/Event), rodar migrate
- **Bloco B — Workers (3-4h):** Criar os 5 workers BullMQ reaproveitando a logica existente de `types.ts`, `validator.ts`, `enricher.ts`, `scoring.ts` e `actions.ts`. O `pipeline.ts` vira apenas o enfileirador.
- **Bloco C — API + Tracking (2-3h):** Refatorar `/api/events` para enfileirar ao inves de processar sincronamente, criar `/api/events/track` e `/api/events/webhook`, criar `lib/eventTracker.ts`

2.2. Decisao 2: Coexistencia vs Migracao

DECISAO: Opcao A — Migracao Direta (Refactor Limpo)

Racional: O plano original mencionava "workers existentes" (decision-worker, marketing-worker, delivery-worker, telemetry-worker) como potencial conflito. Porem, a auditoria confirmou que **nenhum desses workers existe no codebase**. O que existe e um unico pipeline sincrono em `src/lib/events/` que sera refatorado para workers BullMQ. Nao ha codigo para "coexistir". A migracao e um lift-and-shift:

extrair cada estagio do pipeline.ts para um worker dedicado, mantendo intactos os modulos auxiliares (types.ts, scoring.ts, etc.).

Impacto: O endpoint POST /api/events passara de processamento sincrono para assincrono. A resposta sera imediata (202 Accepted) e o processamento real ocorrera nos workers. Isso elimina timeouts, permite retries automaticos em caso de falha, e habilita rate limiting via Redis. O behavior change e transparente para o frontend: ele ja recebe uma resposta e pode consultar os resultados via GET /api/events/stats.

2.3. Decisao 3: ActionLog vs Logs Existentes

DECISAO: Modelo Novo Separado (Nao Integrado)

Racional: O plano original sugeria possivel conflito com AgentLog e SecurityAlert existentes. A auditoria confirmou que **nenhum desses modelos existe** no schema Prisma. Nao ha sistema de logs para conflitar. Criaremos ActionLog e WebhookLog como modelos novos, seguindo exatamente o blueprint do PDF. Eles serao a fundacao de governanca do sistema ZEHLA Brain.

Design: O ActionLog registrou cada acao automatica disparada (email de nurturing, alerta de vendas, ajuste de bid), com leadId, actionType, trigger, cluster, payload JSON e status (pending/sent/failed). O WebhookLog registrara cada webhook recebido de plataformas externas (n8n, Make, Segment), com source, payload, headers e status. Juntos, fornecem rastreabilidade completa de todas as interacoes automatizadas do sistema.

3. Estrategia Consolidada de Implementacao

A estrategia unificada consolida as 5 fases originais em 3 blocos de execucao atomica. Cada bloco e autossuficiente e pode ser testado independentemente. A sequencia e projetada para minimizar codigo morto: cada bloco imediatamente utiliza o que o anterior criou. O reaproveitamento da logica existente em src/lib/events/ e maximo, reduzindo risco e acelerando a entrega.

Acao	Arquivo	Descricao
MODIFY	prisma/schema.prisma	Adicionar campos ao Lead: previousCluster, lastActivityAt, eventSource, fingerprint
MODIFY	prisma/schema.prisma	Adicionar campos ao Event: dedupHash, sessionId, status (pipeline tracking)
NEW	prisma/schema.prisma	Criar modelo ActionLog (leadId, actionType, trigger, cluster, payload, status)
NEW	prisma/schema.prisma	Criar modelo WebhookLog (source, eventType, payload, headers, status)
INSTALL	package.json	npm install bullmq ioredis (dependencias de fila e Redis)
NEW	src/lib/redis.ts	Conexao Redis centralizada com ioredis (retry, readyCheck)

NEW	src/lib/queues.ts	Modulo centralizado: 5 filas BullMQ + QUEUE_NAMES + WORKER_CONFIG
RUN	Terminal	npx prisma migrate dev (aplicar mudancas no schema)
NEW	.env	Adicionar REDIS_URL, WEBHOOK_SECRET e documentar variaveis obrigatorias

Tabela 6: Bloco A — Itens de implementacao

Ponto de reaproveitamento: Os tipos em src/lib/events/types.ts (EventType, Cluster, FunnelStage, score impacts) estao completos e corretos. Nao precisa ser modificado. O modulo scoring.ts com classificarLead(), calcularCluster() e inferirDor() sera reaproveitado integralmente no classifyWorker.

Acao	Arquivo	Descricao
NEW	src/workers/captureWorker.ts	Rate limiting (Redis), priorizacao por tipo de evento, encaminhar para validate
NEW	src/workers/validateWorker.ts	Validar payload, deduplicar (dedupHash), upsert lead, persistir Event
NEW	src/workers/enrichWorker.ts	Geo enrichment (ipapi.co), device parsing, historico do lead (30 dias)
NEW	src/workers/classifyWorker.ts	Reaproveitar scoring.ts: recalcular score, detectar transicao de cluster
NEW	src/workers/actWorker.ts	Action Dispatcher: mapear transicao->acoes, executar, logar no ActionLog
NEW	src/workers/index.ts	Orquestrador: iniciar todos os workers + graceful shutdown (SIGTERM/SIGINT)
MODIFY	src/lib/events/actions.ts	Conectar placeholders reais: email via Resend, WhatsApp via Z-API
MODIFY	src/lib/events/pipeline.ts	Transformar de pipeline sincrona para enfileirador BullMQ

Tabela 7: Bloco B — Itens de implementacao

Fluxo apos Bloco B: POST /api/events recebe payload, adiciona job na captureQueue (202 Accepted), captureWorker valida e encaminha para validateQueue, validateWorker persiste e encaminha para enrichQueue, enrichWorker adiciona contexto e encaminha para classifyQueue, classifyWorker recalcula score/cluster e se houve transicao encaminha para actQueue, actWorker dispara acoes automaticas e registra no ActionLog.

Acao	Arquivo	Descricao
MODIFY	src/app/api/events/route.ts	Substituir pipeline sincrona por captureQueue.add() (202 Accepted)
NEW	src/app/api/events/track/route.ts	Endpoint POST generico: aceita qualquer eventType com score mapping
NEW	src/app/api/events/webhook/route.ts	Webhook receiver: HMAC verification + mapeamento eventos externos

MODIFY	<code>src/app/api/events/stats/route.ts</code>	Adicionar metricas de transicoes de cluster e acoes disparadas
NEW	<code>src/lib/eventTracker.ts</code>	Modulo client-side: trackEvent(), useEventTracker() hook, fingerprint
NEW	<code>src/hooks/useEventTracker.ts</code>	React hook para tracking automatico de page views e cliques

Tabela 8: Bloco C — Itens de implementacao

4. Mapa de Arquivos — Antes e Depois

A tabela abaixo compara a estrutura de arquivos antes e depois da implementacao. Arquivos marcados como REAPROVEITADO serao mantidos com modificacoes minimas. Arquivos NOVOS serao criados do zero. O objetivo e maximizar o reaproveitamento do codigo existente, especialmente os modulos de tipos, validacao e scoring que ja estao funcionais.

Arquivo	Acao	Detalhes
<code>prisma/schema.prisma</code>	MODIFY	Expandir Lead/Event + ActionLog + WebhookLog
<code>src/lib/events/types.ts</code>	REAPROVEITADO	Sem mudancas — tipos, scores, clusters corretos
<code>src/lib/events/validator.ts</code>	REAPROVEITADO	Reutilizado pelo validateWorker
<code>src/lib/events/enricher.ts</code>	REAPROVEITADO	Reutilizado pelo enrichWorker
<code>src/lib/events/scoring.ts</code>	REAPROVEITADO	Reutilizado pelo classifyWorker
<code>src/lib/events/actions.ts</code>	MODIFY	Conectar placeholders a APIs reais
<code>src/lib/events/pipeline.ts</code>	REFACTOR	De pipeline sync para enfileirador BullMQ
<code>src/lib/events/index.ts</code>	REAPROVEITADO	Manter barrel export
<code>src/lib/redis.ts</code>	NOVO	Conexao Redis centralizada
<code>src/lib/queues.ts</code>	NOVO	5 filas BullMQ centralizadas
<code>src/lib/eventTracker.ts</code>	NOVO	Client-side tracking + React hook
<code>src/workers/captureWorker.ts</code>	NOVO	Worker Estagio 1
<code>src/workers/validateWorker.ts</code>	NOVO	Worker Estagio 2
<code>src/workers/enrichWorker.ts</code>	NOVO	Worker Estagio 3
<code>src/workers/classifyWorker.ts</code>	NOVO	Worker Estagio 4
<code>src/workers/actWorker.ts</code>	NOVO	Worker Estagio 5
<code>src/workers/index.ts</code>	NOVO	Orquestrador + Graceful Shutdown
<code>src/app/api/events/track/route.ts</code>	NOVO	Endpoint POST generico

src/app/api/events/webhook/route.ts	NOVO	Webhook receiver externo
.env	MODIFY	Adicionar REDIS_URL, WEBHOOK_SECRET, etc.

Tabela 9: Mapa de arquivos — antes e depois

5. Plano de Verificacao

A verificacao garante que cada bloco foi implementado corretamente antes de avancar para o proximo. Cada bloco tem testes automatizados e verificacao manual especificos. O criterio de aceite para avancar e que todos os testes do bloco atual passem sem falhas.

5.1. Verificacao do Bloco A

- npx prisma migrate dev — schema deve migrar sem erros
- npx prisma generate — cliente Prisma gerado com modelos novos
- Redis connectivity: node -e "require('ioredis').default.connect()" — conexao OK
- Verificar que ActionLog e WebhookLog aparecem no Prisma Studio

5.2. Verificacao do Bloco B

- Script de teste: enviar evento POST /api/events com payload de teste
- Verificar nos logs: [Capture] -> [Validate] -> [Enrich] -> [Classify] -> [Act]
- Consultar GET /api/events/stats e verificar que o evento aparece nas metricas
- Simular transicao COLD->WARM: enviar 4 eventos sequenciais (email_open + link_click + landing_visit + whatsapp_open) e verificar que o lead muda de cluster
- Verificar ActionLog: consultar tabela e confirmar que acoes foram registradas

5.3. Verificacao do Bloco C

- POST /api/events/track com tipo "email_open" — deve retornar 202
- POST /api/events/webhook com payload Segment — deve mapear e processar
- useEventTracker() hook no dashboard — deve trackear page views automaticamente
- Rate limiting: enviar 60 eventos em 1 minuto para mesmo email — 10 devem ser rejeitados

6. Matriz de Riscos

Cada risco foi avaliado por probabilidade e impacto, com estrategia de mitigacao definida. Os riscos foram identificados com base na auditoria do codebase e nas mudancas arquiteturais propostas pela estrategia consolidada.

Risco	Prob.	Impacto	Mitigacao
Redis indisponivel	Media	Alto	BullMQ com retries + fallback para pipeline sincrona
Perda de dados na migracao Prisma	Baixa	Critico	Backup do SQLite antes de migrate + migrate dev (reversivel)
Workers nao processam eventos	Media	Alto	Health check endpoint + metricas de queue depth
Deduplicacao falha com multi-instancia	Baixa	Medio	dedupHash no DB (Prisma unique) garante idempotencia
Rate limiting muito restritivo	Baixa	Medio	Configuravel via env var, 100 eventos/hora como default
SQLite nao suporta concorrencia	Alta	Medio	Migrar para PostgreSQL antes de deploy em producao
Breaking change no POST /api/events	Media	Alto	Mantehave o campo "status" no Event para rastrear pipeline

Tabela 10: Matriz de riscos

Risco Principal: SQLite nao suporta escritas concorrentes. Quando os workers BullMQ processarem eventos em paralelo, havera race conditions no banco. A solucao e migrar para PostgreSQL antes do deploy em producao. Para desenvolvimento local, SQLite funciona porque os workers terao concurrency=1.

7. Timeline Estimada

A timeline considera um desenvolvedor experiente trabalhando em ritmo focado. Os blocos sao sequenciais (B depende de A, C depende de B) mas os itens dentro de cada bloco podem ser paralelizados quando nao ha dependencias entre si.

Bloco	Duracao	Itens	Entregavel
A: Fundacao	2-3 horas	9 itens	Prisma migrado, Redis conectado, 5 filas criadas
B: Workers	3-4 horas	8 itens	Pipeline assincrona operacional, acoes logadas
C: API + Tracking	2-3 horas	6 itens	Endpoints + React hook funcionais
Verificacao	1-2 horas	12 testes	Todos os criterios de aceite passando

Tabela 11: Timeline estimada por bloco

Total estimado: 8-12 horas de trabalho focado. O reaproveitamento dos modulos existentes (types.ts, validator.ts, enricher.ts, scoring.ts) reduz significativamente o esforco de desenvolvimento. Sem esse reaproveitamento, a estimativa seria de 20-30 horas.